Securing a Small Self-Hosted Server

Layered defences against ordinary and AI-driven attacks, and what changes when the server runs an AI agent

Davide Maccabelli
Independent project

October 2026 — short technical note

Abstract

I run a small Linux server on a free cloud tier for personal projects, including a self-hosted AI assistant. This note describes how I protect it and why. The core idea is defence in layers: expose as little as possible, detect and block what still gets through, give every account and program only the rights it needs, keep software and models up to date and from trusted sources, and keep backups that the server itself cannot destroy. On my server, every inbound port is closed except a single access point, I connect through a WireGuard VPN, and fail2ban and CrowdSec watch the logs and block abusive addresses. Reviewing my own setup plan for the AI assistant, I found that two permissions which looked limited were in fact equivalent to full administrator rights, and that the assistant's safety rules were only text given to the model. Those findings are now on my list to fix before the assistant gets any command access. AI changes the picture in two ways. Attackers can now automate reconnaissance and exploitation, which shortens the time between a public vulnerability and an attack on my server. And an AI agent with tools is itself an attack surface: text it reads can turn into instructions. The defences against both are mostly old principles applied more strictly. On purpose, this note contains no addresses, port numbers, account names or configuration files.

1 Why a small server needs this

A personal server is not an interesting target for a skilled attacker, but it does not need to be. As soon as a machine has a public address, automated programs probe it for open services, weak passwords and known vulnerabilities, all day and without choosing their victims. A compromised server is useful to them as it is: for sending spam, mining cryptocurrency, attacking others or, for an AI server, reselling its computing power. My server runs on a free ARM instance from a public cloud provider. I planned to use it for open-source language models, an assistant able to run commands, a few containers and layered backups. That plan raised the stakes: a program that executes commands is exactly what an attacker would like to control.

I wrote this note to put my choices in order, to check them against current guidance, and to keep the reasoning for when I build the next server. Table 1 lists what I am protecting and from whom.

2 Layer 1: expose as little as possible

Anything that cannot be reached cannot be attacked from outside. This is the most effective layer, and the cheapest.

Closed by default. The cloud provider's network rules and the firewall on the server both deny all inbound traffic by default. On my server, every port is closed except the single one I use to reach the server myself. Two independent firewalls matter, because a mistake in one is caught by the other.

Threat	What it looks like for my server
Automated scanning	Bots that try every port and every common password, and exploit known vulnerabilities in exposed services.
AI-assisted attackers	The same bots, faster: tools that read a vulnerability report and write a working exploit, or that map exposed AI services.
Stolen credentials	A key or password taken from my laptop, a phishing page or a leaked file, used to log in normally.
Supply chain	A package, container image or model file that runs malicious code when installed or loaded.
My own AI agent	An assistant that follows instructions hidden in a document or web page, or that has more rights than its task needs.
My own mistakes	A port opened "for a test", a setting left at its default, a backup that was never tested.

Table 1: Threat model. Assets: the server and its data, my backups, the cloud account that controls the server, and the keys I use to reach it.

A VPN as the only door. I connect through WireGuard. Its design has a property that is very useful here: a WireGuard server does not respond at all to packets that do not carry a valid key, so to a scanner the port looks closed (WireGuard, 2026). The best use of the single open port is therefore the VPN, with remote administration (SSH) reachable only from inside the tunnel, with keys instead of passwords and with direct administrator login disabled.

Services listen only where they must. A database, a dashboard or a language-model API should listen on the local interface or on the VPN, never on all interfaces. Two traps are worth knowing. First, Docker writes its own packet-filtering rules: when a container port is published, "traffic to and from that container gets diverted before it goes through the ufw firewall settings" (Docker, 2026b). A container can therefore be reachable from the internet even when the host firewall says it is closed, unless the port is bound to a local address or the cloud rules block it. Second, Ollama, a popular way to run local models, has no authentication by default. In a scan reported in May 2026, 31% of more than 5,200 Ollama servers reachable from the internet answered test prompts from anyone; honeypots recorded over 91,000 attack sessions against exposed language-model endpoints between October 2025 and January 2026 (CSA, 2026). A model API must never be exposed directly.

3 Layer 2: detect and slow down

fail2ban reads the logs of services such as SSH and bans, for a while, the addresses that fail to log in too many times (Fail2Ban, 2026).

CrowdSec does something similar with richer behaviour rules, and adds a shared element: installations report the attacks they see and receive a community blocklist of addresses already identified as malicious (CrowdSec, 2026). I run both. With only the VPN exposed, they have little to see; their value is as a second line, in case a service is exposed by mistake or the configuration changes. Neither protects against an attacker who has a valid key, because a valid login is not suspicious. That is why the following layers matter.

4 Layer 3: least privilege

The rule is simple: every account and every program gets only the rights its task requires. It is easy to state and easy to get wrong, because some permissions that look narrow are not. Reviewing the setup guide I had written for the AI assistant on my server, I found three problems:

- **A group that equals administrator rights.** The guide put the assistant's account in the `docker` group so that it could manage containers. Docker's documentation states that this group grants root-level privileges (Docker, 2026a): whoever can start a container can mount the host's disk inside it.
- **An installation right that equals every right.** The guide let the same account install packages with administrator rights, with any argument. Package managers accept options and configuration that can run arbitrary commands, so "may install any package" amounts to "may do anything". A permission is only as narrow as the most powerful thing it can be used for.
- **Safety rules that are only text.** The guide gave the assistant a file of rules, such as never deleting system files. These rules are useful guidance for the model, but they are not enforcement: a model can ignore them, and a document it reads can contradict them.

Two smaller points came out of the same review: a backup script that passed a database password on the command line, where other users of the machine can see it in the process list, and an installation step that ran a downloaded script without checking it. None of these is fixed yet. They are the first items on my list, and the assistant will not get command access on the server until they are.

5 Layer 4: updates and supply chain

Updates. Security updates for the operating system should install automatically, followed by a restart of the services that need it. I still have to check that this is active on my server. The reason is speed, as Section 7 explains: AI tools make it faster to turn a published vulnerability into an attack.

What I install. Packages should come from the distribution and container images from official publishers, with a fixed version. Commands that pipe a script from the internet straight into a shell are best avoided; my own setup guide used one, as noted above.

Model files are code. A model is not just data. Checkpoints saved with Python's pickle format can execute code when they are loaded. PyTorch made its safer loading mode the default in version 2.6, but in 2026 a flaw was disclosed through which a crafted file could corrupt memory, and potentially run code, even in that mode; it was fixed in version 2.10 (CVE-2026-24747, 2026). My rule for the AI server is therefore to use formats that store only numbers (GGUF and safetensors), to download models only from known publishers, to check their hashes and to keep the loading libraries up to date.

6 Layer 5: backups the server cannot destroy

Backups protect against hardware failure, my own mistakes, ransomware and an assistant that deletes the wrong thing. To protect against the last two, the server must not be able to delete or overwrite its own backups: the copies are pulled by another machine, or written to storage that keeps old versions and refuses deletions for a period. At least one copy lives outside the cloud account, encrypted. A backup counts only after a restore has been tested.

7 What AI changes

7.1 Attackers with AI

Two pieces of evidence convinced me that this is not hypothetical. In a 2024 study, an agent built on GPT-4 exploited 13 of 15 real "one-day" vulnerabilities (87%) when given the public description of the vulnerability, and only 7% without it (Fang et al., 2024). In other words,

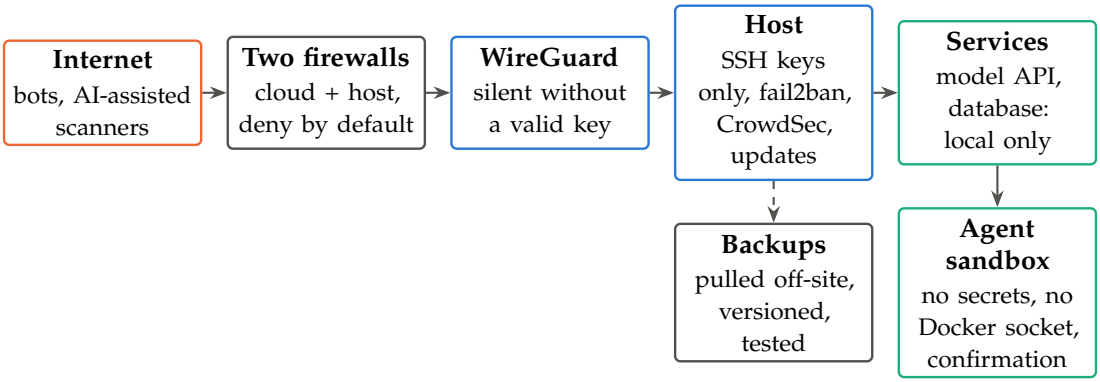


Figure 1: The layers, from the internet inwards. Each one assumes that the one before it can fail.

a published security advisory is close to a ready-made attack. In November 2025 Anthropic reported a state-sponsored espionage campaign, detected in September, in which an AI coding agent carried out an estimated 80–90% of the tactical work against about 30 targets, with humans approving the key steps (Anthropic, 2025). The same report notes a limit: the AI often overstated its findings and sometimes invented them.

For a small server the consequences are practical. The window between a vulnerability becoming public and being exploited shrinks, so updates must be automatic and exposure minimal. Phishing gets more convincing, so the cloud account, which controls everything, needs strong multi-factor authentication, ideally a passkey or hardware key. And scanning for exposed AI services is already systematic, as the numbers in Layer 1 show.

7.2 My own agent as an attack surface

Giving an AI assistant tools turns it into a new kind of user of my server, one that can be talked into things. The OWASP Top 10 for LLM applications names the main risks (OWASP, 2025). The ones that matter most here are:

- **Prompt injection:** instructions hidden in content the model reads (a web page, a document, an email, a memory) are followed as if I had written them.
- **Excessive agency:** the agent has more functions, rights or autonomy than its task requires, so a single mistake or injection does real damage.
- **Vector and embedding weaknesses:** a poisoned document in the memory store is retrieved later and enters the prompt.
- **Supply chain** and **unbounded consumption:** malicious models or plug-ins, and requests that exhaust the machine.

A useful way to think about it is the "lethal trifecta" (Willison, 2025): an agent that combines access to private data, exposure to untrusted content and a way to communicate outwards can be tricked into sending the data to an attacker. Removing any one of the three breaks the attack.

The defences follow from one principle: *the model is never the security boundary*. In practice, for the command-execution phase of my assistant:

- commands run in a disposable container or virtual machine, with no secrets, no access to the Docker socket, no network unless the task needs it, and limits on memory and time;
- an allowlist of commands and arguments, without wildcards, and my explicit confirmation, showing the exact command, for anything that changes the system or sends data out;
- retrieved text is marked as data, with its source, and never treated as an instruction; memories from untrusted sources are stored separately;
- the agent never has private data, untrusted content and outbound network access at the same time;

- every action is logged, and a backup is taken before risky operations.

8 Checklist

Table 2 summarises the controls and where I stand. I mark as "in place" only what I have configured on my server; "to verify" means I believe it is set but have not checked it recently; "planned" means it is not done.

Control	Protects against	Status
All inbound ports closed except one access point	Scanning, exploitation of exposed services	In place
WireGuard VPN for access	Scanning, password attacks	In place
fail2ban and CrowdSec	Repeated login attempts, known bad addresses	In place
SSH only through the VPN, keys only, no direct administrator login	Stolen or guessed passwords	To verify
No container port published on public interfaces	Docker bypassing the host firewall	To verify
Model API and database on local interfaces only	Exposed AI endpoints, data theft	To verify
Assistant account without Docker group or broad installation rights	Privilege escalation	Planned
Database password out of the command line	Credential exposure	Planned
Automatic security updates	Fast exploitation of public vulnerabilities	To verify
Model files in GGUF or safetensors, from known publishers, with hashes	Malicious model files	Planned
Sandbox, allowlist and confirmation for agent commands	Prompt injection, excessive agency	Planned
Off-site, versioned backups the server cannot delete, restore tested	Ransomware, deletion, my mistakes	Planned
Passkey or hardware key on the cloud account	Phishing, account takeover	Planned

Table 2: Controls and status as of October 2026.

9 Conclusion

Most of what protects a small server against AI-driven attacks is what protected it before: expose almost nothing, patch quickly, give each part the least power it needs and keep backups out of reach. AI makes these rules less optional, because attackers automate faster and because an AI agent with tools is a new kind of insider that can be manipulated through the text it reads. The most useful thing I did was to review my own plan as if someone else had written it: it showed me that two permissions that looked harmless were full administrator rights. My next step is to fix the items marked "planned" and only then give the assistant access to commands, starting inside a disposable sandbox.

What this note leaves out. I deliberately omit my server's address, the port I use, account names and configuration files. Publishing them would help an attacker and teach a reader nothing that the principles above do not.

References

Anthropic (2025). Disrupting the first reported AI-orchestrated cyber espionage campaign. Report, November 2025.

Cloud Security Alliance (2026). Exposed AI infrastructure: self-hosted services under attack. Research note, 7 May 2026 (reporting data from Intruder and GreyNoise).

CrowdSec (2026). Security Engine overview. `https://docs.crowdsec.net/docs/intro`.

CVE-2026-24747 (2026). PyTorch: the `weights_only` unpickler fails to validate pickle opcodes and storage metadata; fixed in PyTorch 2.10.0.

Docker Inc. (2026a). Linux post-installation steps for Docker Engine: manage Docker as a non-root user. `https://docs.docker.com/engine/install/linux-postinstall/`.

Docker Inc. (2026b). Packet filtering and firewalls. `https://docs.docker.com/engine/network/packet-filtering-firewalls/`.

Fail2Ban project (2026). `https://github.com/fail2ban/fail2ban`.

Fang, R., Bindu, R., Gupta, A., Kang, D. (2024). LLM agents can autonomously exploit one-day vulnerabilities. arXiv:2404.08144.

OWASP Foundation (2025). OWASP Top 10 for Large Language Model Applications, 2025 edition.

Donenfeld, J. A. WireGuard: protocol and cryptography. `https://www.wireguard.com/protocol/`.

Willison, S. (2025). The lethal trifecta for AI agents: private data, untrusted content, and external communication. Blog post, 16 June 2025.